

1. TITLE PAGE

Course Name: PCCCS405: Advanced Programming (OOP) Lab

Project Title: Smart Tasker – Task Management System

Student Name: Kamakshya Ghosh

Roll Number: 29

Instructor Name: Soumadip Biswas

Semester: Spring 2026

2. PROBLEM STATEMENT

In today's fast-paced academic and professional environments, effective task management has become an essential requirement for maintaining productivity and meeting deadlines. Students and working professionals often handle multiple tasks simultaneously, including assignments, project work, meetings, and personal responsibilities. However, many individuals struggle to organize and track these tasks efficiently, leading to missed deadlines, reduced productivity, and increased stress. The absence of a simple, reliable, and accessible system for managing daily tasks is a common issue faced by users.

Although there are numerous task management applications available, many of them are overly complex, require constant internet connectivity, or include features that are unnecessary for users seeking a basic solution. Additionally, such applications may involve steep learning curves, advertisements, or subscription-based models, which make them less suitable for quick and straightforward task tracking. As a result, there is a clear need for a lightweight, efficient, and user-friendly system that focuses on the fundamental aspects of task management without unnecessary complications.

The objective of this project is to design and develop a console-based application called Smart Tasker using the Java programming language. The system aims to provide users with a simple interface through which they can create, view, update, and delete tasks. It also allows users to mark tasks as completed, enabling them to track progress effectively. The application maintains task data using file handling techniques, ensuring that tasks are saved and can be retrieved even after the program is restarted.

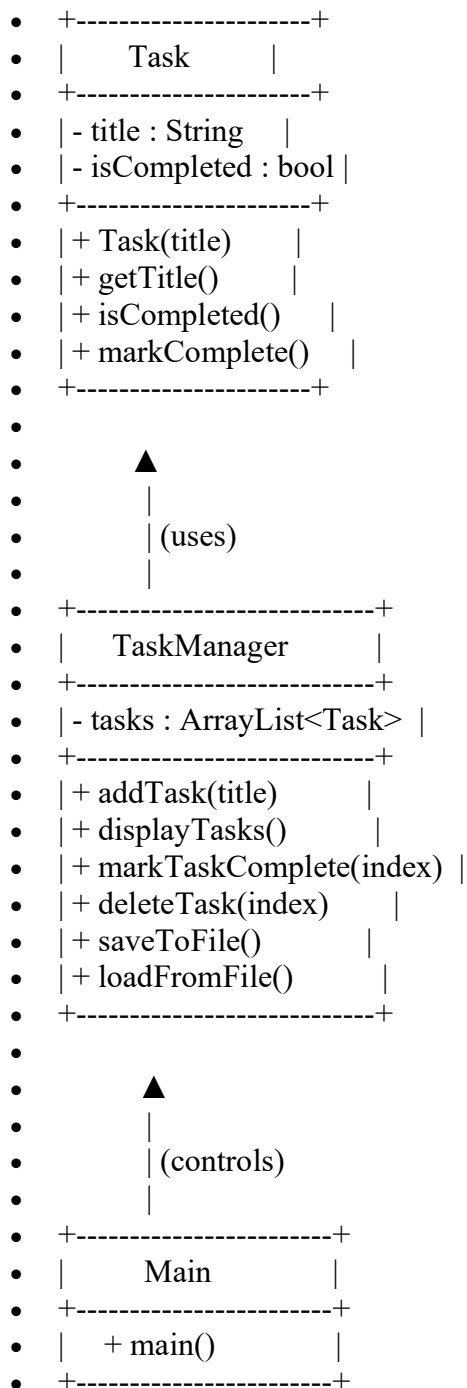
The Smart Tasker system is designed with a focus on strong software engineering principles, particularly Object-Oriented Programming (OOP). It uses encapsulation to protect data within classes, abstraction to simplify interactions with the system, and modular design to separate responsibilities across different components such as task representation, task management, and user interaction. This structured approach ensures that the application is maintainable, scalable, and easy to extend in the future.

Furthermore, the project emphasizes robustness and reliability by incorporating exception handling mechanisms to manage invalid user inputs and unexpected runtime errors. This ensures that the application does not crash and provides meaningful feedback to the user in case of incorrect operations.

Overall, the Smart Tasker project addresses the problem of inefficient task management by providing a clean, minimal, and functional solution. It demonstrates how core programming concepts can be applied to solve real-world problems effectively while maintaining code quality and design clarity. The system is particularly useful for individuals who require a simple yet reliable tool for managing their daily activities without the complexity of advanced applications.

3. SYSTEM DESIGN

3.1 UML Class Diagram (Description)



3.2 Description of Major Classes

Task Class

- Represents a single task
- Attributes:
 - title
 - completion status
- Methods:
 - markComplete()
 - getters

TaskManager Class

- Manages all tasks
 - Uses ArrayList<Task>
 - Handles:
 - add task
 - delete task
 - display tasks
 - mark complete
 - file saving/loading
-

Main Class

- Entry point of program
 - Handles user input
 - Provides menu-driven interface
-

3.3 Package Structure

```
src/  
├── Main.java  
├── model/  
│   └── Task.java  
└── service/  
    └── TaskManager.java
```

3.4 Interaction Flow

1. User selects option from menu
 2. Input is read using Scanner
 3. Request sent to TaskManager
 4. TaskManager updates task list
 5. Output displayed to user
-

4. OOP CONCEPTS DEMONSTRATED

4.1 Encapsulation

Where used: Task class

Why: Protect data and control access

Code Snippet:

```
private String title;  
private boolean isCompleted;
```

```
public String getTitle() {  
    return title;  
}
```

Explanation:

Data is hidden using private variables and accessed via getter methods.

4.2 Abstraction

Where used: TaskManager methods

Code Snippet:

```
public void addTask(String title) {  
    tasks.add(new Task(title));  
}
```

Explanation:

User does not see internal implementation of task storage.

4.3 Polymorphism (Conceptual / Extendable)

Where used: Can be extended using different task types

Explanation:

System design allows extension into multiple task types in future.

4.4 Exception Handling

Where used: User input

Code Snippet:

```
try {  
    int choice = sc.nextInt();  
} catch (Exception e) {  
    System.out.println("Invalid input");  
}
```

Explanation:

Prevents program crash due to invalid input.

5. IMPLEMENTATION HIGHLIGHTS

Snippet 1: Add Task

```
public void addTask(String title) {
    tasks.add(new Task(title));
}
```

Justification:
Ensures tasks are dynamically added using ArrayList.

Snippet 2: Display Tasks

```
for (int i = 0; i < tasks.size(); i++) {
    System.out.println(tasks.get(i).getTitle());
}
```

Justification:
Provides indexed view for user operations.

Snippet 3: Mark Complete

```
tasks.get(index).markComplete();
```

Justification:
Efficient update of task state.

Snippet 4: File Save

```
FileWriter fw = new FileWriter("tasks.txt");
fw.write(task.getTitle());
```

Justification:
Enables persistence of data.

6. TESTING & ERROR HANDLING

Test Cases Considered

<i>Case</i>	<i>Expected Result</i>
Add task	Task added
Delete task	Task removed
Invalid index	No crash
Empty list display	Proper message

Edge Cases

- No tasks available
 - Invalid user input
 - File not found
-

Failure Scenarios

- Input mismatch → handled using try-catch
 - File errors → handled gracefully
-

7. GIT WORKFLOW

Commit Strategy

- 12+ meaningful commits
 - Each commit represents small progress
-

Development Progression

1. Project initialization
 2. Task class creation
 3. TaskManager implementation
 4. Menu system
 5. Feature addition
 6. File handling
 7. Exception handling
-

Evidence

- Git commit history screenshot attached

8. CONCLUSION & FUTURE SCOPE

Conclusion

The Smart Tasker project successfully demonstrates the application of Object-Oriented Programming principles in Java. It provides a simple yet effective system for task management while maintaining clean architecture and modular design.

Future Scope

- GUI (JavaFX / Swing)
 - Task priority system
 - Reminder notifications
 - Database integration
 - Multi-user support
-

APPENDIX

Project Title

Smart Task Scheduler with Priority-Based Execution in Java

Problem Statement

Students and professionals frequently manage multiple assignments, deadlines, and responsibilities simultaneously. Traditional to-do list applications rely on manual prioritization and static ordering of tasks, which often leads to inefficient scheduling when deadlines change or workload increases. Users must constantly reorganize their task lists, resulting in missed deadlines and poor time management.

The proposed **Smart Task Scheduler** addresses this limitation by automatically organizing tasks based on priority, deadline, and estimated duration. The system will dynamically reorder tasks using a priority-based scheduling mechanism implemented with Java collections. Tasks will be modeled using an object-oriented hierarchy where common behaviors are abstracted and specialized task types extend the base functionality. The application will demonstrate strong object-oriented design through abstraction, inheritance, polymorphism, and modular architecture while providing efficient task management and improved productivity for users.

Target User

College students and early-career professionals who manage multiple academic, personal, or project-related tasks.

Core Features

- Add, update, and delete tasks
- Automatic priority-based task scheduling
- Deadline-based task reordering
- Support for recurring and one-time tasks
- Task completion tracking
- Persistent storage of tasks using file handling
- Robust input validation and exception handling
- Dynamic task queue using PriorityQueue

OOP Concepts to be Used

Abstraction

An abstract **Task** class will define common attributes such as title, deadline, priority, and duration.

Inheritance

Specialized subclasses like **RecurringTask** and **OneTimeTask** will extend the Task class.

Polymorphism

Task priority calculation and scheduling behavior will be overridden by different task types.

Exception Handling

Custom exceptions will handle invalid inputs, deadline conflicts, and incorrect task operations.

Collections

Java **PriorityQueue** will be used to dynamically reorder tasks based on scheduling priority.

Proposed Architecture Description

The system will follow a **layered object-oriented architecture**. An abstract **Task** class will define common behavior for all tasks, while specialized subclasses implement different task types. A **SchedulerManager** class will manage task creation, priority evaluation, and scheduling using a PriorityQueue. A separate **PersistenceManager** will handle saving and loading tasks from file storage. This separation ensures modular design, maintainability, and clear separation between domain logic, scheduling behavior, and data persistence.

Architecture

Task (Abstract Class) → Specialized Task Classes → SchedulerManager → Persistence Layer

Flow:

PCCCS495 Proposal Review – APPROVED_MINOR

From: soumadip.biswas@ieminternational.ai

soumadip.biswas@ieminternational.ai

To: Kamakshya Ghosh

Kamakshya.Ghosh2024@iem.edu.in

Sent: Tuesday 10 March at 4:04 pm

Dear Kamakshya Ghosh,

Project: Smart Task Scheduler with priority Based
Execution

Decision: APPROVED_MINOR

Score: 90

Strengths:

The proposal presents a clear and relatable problem statement, directly addressing inefficiencies in traditional task management. It proposes a well-structured solution utilizing core Java OOP principles (Abstraction, Inheritance, Polymorphism, Exception Handling, Collections) and a highly appropriate data structure (`PriorityQueue`) for dynamic scheduling. The layered architecture (Task hierarchy, Scheduler Manager, Persistence Manager) demonstrates a strong understanding of modular design and separation of concerns. The features list is comprehensive and practical, suitable for a university-level project. Explicit mention of robust input validation and exception handling is a positive sign for application stability.

Improvements:

While the proposal is strong, a few minor clarifications would enhance it further:

1. ****Scheduling Logic Detail:**** Elaborate slightly on the specific criteria and logic used to determine task priority for the `PriorityQueue` (e.g., a custom `Comparator` that considers deadline, user-assigned importance, and estimated duration).
2. ****Persistence Format:**** Specify the intended file storage format (e.g., plain text, CSV, JSON, or Java object serialization) to provide more detail on the 'Persistence Manager'.
3. ****User Interface:**** Briefly mention the intended user

interface (e.g., a simple command-line interface, or if a GUI framework like Swing/JavaFX is considered), as this impacts user interaction, even if the core OOP focuses on the backend.

4. ****Task Type Examples:**** Provide one or two concrete examples of 'Specialized Task Classes' (e.g., `AcademicTask`, `WorkTask`) to better illustrate the inheritance hierarchy in action.

Regards,
Course Instructor